LLM Fingerprinting: RQ2 and RQ4

CMPSC 448 | Allegra Francis contribution supplement | October 11, 2026

Uses Amal's current dataset and shared model/training implementation. Combine with the team's RQ1/RQ3 work and final leader/member identification. This supplement is not the complete team submission.

Exact shared dataset

The existing data/processed/raid_balanced.csv contains 1,600 responses from chatgpt, llama-chat, mistral-chat and cohere-chat, representing four distinct families. Domains are news, poetry, recipes and reviews. Each domain has 100 matched original generation prompts, each answered by all four models. The team builder uses RAID-train, attack=none, sampling decoding and no repetition penalty. We reused the CSV unchanged rather than collecting or resampling data.

Domain	Train prompts	Validation	Test prompts	Responses
news	80	10	10	400
poetry	80	10	10	400
recipes	80	10	10	400
reviews	80	10	10	400

The saved 80/10/10 source split gives 1,280 training, 160 validation and 160 test responses. Matched sources and their identical prompts stay in one split. The file hash matches the partner manifest: f95ce226d32562d0f8b7e18946931b20be61ecdeab47d13af32a15c580685ca4. No dataset, source builder, common model code or existing RQ1/RQ3 files were changed.

Reused CNN/LSTM and controlled RQ2 protocol

Import common.train.TrainConfig and train_experiment. Reuse TextCNN with 128-dimensional embeddings, 128 channels per window 3/4/5 and masked max pooling. Reuse the one-layer 128-unit TextLSTM with masked mean pooling. Both use dropout 0.3 and a four-class linear head. AdamW lr 0.001, weight_decay 0.0001, batch 32, clipping 1.0, seed 42, up to 15 epochs and patience 4. Select maximum validation macro-F1, breaking ties with lower validation loss; then evaluate test.

On copies of the rows, preserve the first 128 prompt tokens and first 256 response tokens using the shared tokenizer. Output-only and combined conditions see exactly the same response tokens. Combined adds the prompt and shared newline separator, with maximum 385 tokens. Fit vocabularies separately on visible training text only (max 20,000, min frequency 2). Metadata and labels are excluded from input.

The original RQ1 fits its vocabulary on full training outputs before encoding truncation. Our output-only baseline refits vocabulary on visible output windows for the controlled comparison. Therefore, RQ2 scores are independently measured, not copied from RQ1. Reuse refers to code/architecture, not previously trained weights. Runs here use CPU; numerical results can differ across devices.

RQ2: Does the prompt help overall?

Model	Input condition	Accuracy	Macro-F1
CNN	input	25.0%	0.198
LSTM	input	25.0%	0.192
CNN	output	73.1%	0.733
LSTM	output	53.8%	0.519
CNN	both	70.0%	0.703
LSTM	both	52.5%	0.516

CNN: adding the prompt changes accuracy from 73.1% to 70.0% (-3.1 percentage points), and macro-F1 from 0.733 to 0.703. The paired accuracy-gain interval is [-11.2, +3.8] percentage points.

LSTM: adding the prompt changes accuracy from 53.8% to 52.5% (-1.2 percentage points), and macro-F1 from 0.519 to 0.516. The paired accuracy-gain interval is [-8.7, +6.2] percentage points.

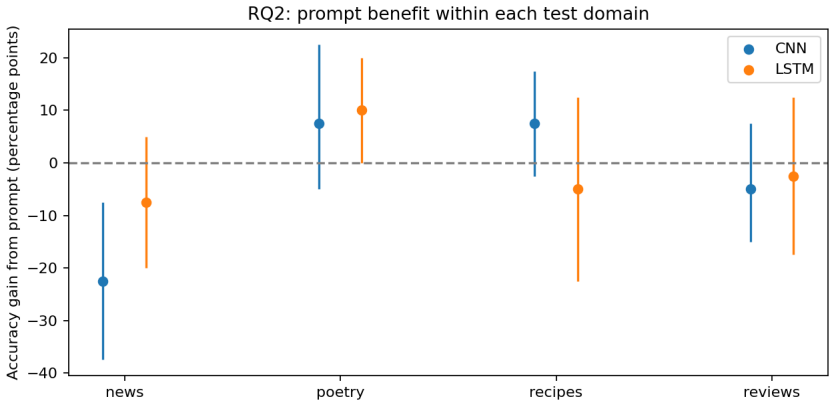
There is no clear overall prompt benefit in this single-seed run: both point estimates decrease, and both overall accuracy-gain intervals include zero. This does not establish that prompts can never help.

Input-only accuracy is exactly 25% because every prompt is paired with all four labels. This is a controlled-design property, not failed training. Macro-F1 need not be 25%. This dataset does not measure users' deliberate platform choices.

Does prompt benefit differ by domain?

Domain	CNN accuracy gain [95% CI], pp	LSTM accuracy gain [95% CI], pp
news	-22.5 [-37.5, -7.5]	-7.5 [-20.0, +5.0]
poetry	+7.5 [-5.0, +22.5]	+10.0 [+0.0, +20.0]
recipes	+7.5 [-2.5, +17.5]	-5.0 [-22.5, +12.5]
reviews	-5.0 [-15.0, +7.5]	-2.5 [-17.5, +12.5]

Poetry improves in both point estimates; CNN recipes also improve. CNN news falls by 22.5 percentage points, with a negative paired interval. Other changes are uncertain. These are exploratory within-run observations, not established universal domain effects.



Gain means input+output minus output-only. Intervals use 2,000 paired bootstrap resamples of entire matched source groups, seed 42. There are 40 test responses but only 10 independent prompts per domain. Intervals are conditional on fixed trained models and exclude training-seed variability. We did not formally test differences between domain gains or adjust multiple comparisons. Domain slices are not RQ3 domain-held-out transfer.

RQ4: Linguistic and structural differences

Descriptive analysis uses full original training responses only, covering length, vocabulary diversity, punctuation, newlines, bullets, headings, code fences and uppercase fraction. Sentence splitting is heuristic. Frequent-word tables are descriptive frequency counts, not neural feature attribution.

Model	Mean words	Raw TTR	First100 TTR	Newlines /100 words
chatgpt	320.4	0.550	0.703	7.95
cohere-chat	186.9	0.595	0.666	8.41
llama-chat	270.0	0.530	0.677	8.42
mistral-chat	211.7	0.566	0.674	8.66

Raw type-token ratio depends on length. First100 fixes its word budget but excludes shorter responses; usable counts appear in feature_summary.csv. Domain-conditioned summaries help inspect task effects. These full-output characteristics need not match the classifier's truncated window.

Retraining ablations with shared models

Model	Condition	Accuracy	Macro-F1	F1 drop
CNN	baseline	73.1%	0.733	+0.000
LSTM	baseline	53.8%	0.519	+0.000
CNN	formatting_removed	69.4%	0.693	+0.040
LSTM	formatting_removed	55.6%	0.539	-0.020
CNN	token_order_shuffled	51.9%	0.497	+0.236
LSTM	token_order_shuffled	60.6%	0.592	-0.073

Shuffling lowers CNN accuracy from 73.1% to 51.9%, but increases LSTM accuracy from 53.8% to 60.6%. Removing formatting lowers CNN accuracy to 69.4% and increases LSTM accuracy to 55.6%. Ordered patterns appear useful for the CNN under this protocol; these results do not support order as essential for the retrained mean-pooling LSTM. Optimization and single-seed variability remain possible explanations.

First select the baseline's visible 256 response tokens. Formatting removal preserves the visible word content and capitalization while removing punctuation/newlines; it does not introduce additional response content. Token shuffling preserves the visible token multiset, with a deterministic per-response seed. Each condition refits training vocabulary and retrains both models using the original split and shared hyperparameters. Negative drops mean improvement.

A performance decrease suggests the altered signals were useful under this training protocol, but these are combined interventions: shuffling changes syntax, local phrases and line layout; formatting removal changes token counts. Descriptive differences do not establish classifier reliance. Single-seed ablation differences are exploratory rather than causal or statistically established.

Verification and scope

All 21 existing and added tests passed. Added checks verify token-budget equivalence, metadata/split preservation and ablation token/word preservation. The setup checker validates the unchanged partner dataset/shared code against the completed run. Performance remains limited to one representative model per family, four domains, fixed generation settings, truncated text and one training seed.

Reproduce or add the completed contribution

The delivery ZIP contains only rq2, rq4, tests/test_rq2_rq4.py, report/allegra_RQ2_RQ4 and START_HERE_ALLEGRA.md. It does not replace data, src/common, scripts, rq1, rq3, root README or requirements. Completed result tables/plots are included, so rerunning is optional. Checkpoints are excluded from the ZIP and ignored by the team's .gitignore.

From the cloned repository root:
`python3 rq2/check_setup.py`
This uses only standard Python libraries to compare dataset and shared-code hashes. See START_HERE_ALLEGRA.md for exact cloning, branching, unzip, commit and pull-request commands.

Optional reproduction after installing the repository requirements:

```
python rq2/run_rq2.py --device cpu --output-dir rq2/results/reproduction
python rq4/run_rq4.py --device cpu --ablations --output-dir rq4/results/reproduction
```

Main files: rq2/results/seed42/summary.csv, prompt_benefit.csv, domain_scores.csv; rq4/results/seed42/feature_summary.csv, features_by_domain.csv, ablation_summary.csv. Per-run directories follow Amal's test_metrics.json, test_predictions.csv, config.json, vocabulary.json, history.json, confusion_matrix.png and learning_curves.png format.

Lessons supported by these experiments

Matched sampling makes the input-only control interpretable. Source-group splitting prevents repeated-prompt leakage. Fixed component budgets avoid confounding prompt addition with lost or extra response evidence. Code reuse improves team consistency, but vocabulary protocol/device differences still require disclosure. Small domain samples yield uncertain estimates, and ablations can change several signals at once. Add or confirm personal lessons using the team's actual experience.

Final team report and disclosure

The assignment requires RQ1 and RQ2, with RQ4 optional extra credit. Combine this contribution with the team's RQ1/RQ3 analysis, full member names, leader designation and verified lessons. The final PDF should cover dataset curation, CNN/RNN and training, results, in-depth analyses and lessons/experience. This supplement does not replace the final team PDF.

OpenAI Codex and ChatGPT assisted with implementation, execution, testing and documentation. Team members must understand, verify and explain all submitted work.

Sources

Team repository: https://github.com/amalsony/llm-fingerprinting
Base commit: 2850ea1e9503695d19fc63780b996a40198356d4
RAID dataset: https://huggingface.co/datasets/liamdugan/raid
Dugan et al. (2024), RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors. https://aclanthology.org/2024.acl-long.674/
Retain the team's dataset provenance and applicable license notices. No human reference documents or private chats were added.